Step 3 — Pause, Resume, and the CLI — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — compile with a checkpointer, detect the pause
- Hint 2 — resume with Command

Hint 1 — compile with a checkpointer, detect the pause

`interrupt()` can only pause against a checkpointer:

```
app = build_graph(model, collection, cutoff).compile(checkpointer=MemorySaver())
thread = {"configurable": {"thread_id": "cli-session"}}
result = app.invoke(start_state(question), thread)

if app.get_state(thread).next:   # non-empty -> it paused at escalate
    ... escalation ...
else:
    print(result["answer"])      # it answered from the docs
```

`app.get_state(thread).next` is `("escalate",)` when the graph paused, and empty when it finished — that's how you tell "answered" from "escalated".

Hint 2 — resume with Command

When it paused, collect a human reply and feed it back:

```
human = input("[human agent]: ")
result = app.invoke(Command(resume=human), thread)   # same thread_id
# now result["answer"] == human, result["escalated"] == True
```

`Command(resume=value)` delivers `value` to the paused `interrupt()`, and the graph runs to the end. Use the **same** `thread_id` — that's how it finds the paused run. For a pause that survives a restart, compile with `SqliteSaver` instead of `MemorySaver`. The integration test checks both

paths: a covered question is answered; an off-topic one pauses and, on resume, finishes with `escalated = True`. The whole file set is in the Solution.